ECE369A Lecture 5 – Quiz1: 20min, 1 submission per table, include all your names

MIPS code given below implements the tomato() function declared as:

```
int tomato(int*  A, int* B, int i, int j)
```

/* A in $a0, B in $a1, i in $a2, j in $a3 */

Write the single line C equivalent expression for this using **`A, B, i`** and **`j`** variables (single expression such as "f=A[B[k]];" from lecture 3).

```
sll  t0, a2, 2
add  t0, a0, t0
sll  t1, a3, 2
add  t1, a1, t1
lw   s0, 12(t0)
addi t2, t0, -16
lw   t0, 4(t2)
add  t0, t0, s0
sw   t0, 8(t1)
```

MIPS code given below implements the tomato() function declared as:

```
int tomato(int*  A, int* B, int i, int j)
```

/* A in $a0, B in $a1, i in $a2, j in $a3 */

Write the single line C equivalent expression for this using **A, B, i** and **j** variables (single expression such as "f=A[B[k]];" from lecture 3).

```
sll  t0, a2, 2      // t0 = 4i
add  t0, a0, t0     // t0 = A+4i = &A[i]
sll  t1, a3, 2      // t1 = 4j
add  t1, a1, t1     // t1 = B+4j = &B[j]
lw   s0, 12(t0)     // s0 = A[i+3]
addi t2, t0, -16    // t2 = A+4i-16 = &A[i-4]
lw   t0, 4(t2)      // t0 = A[i-3]
add  t0, t0, s0     // t0 = A[i-3] + A[i+3]
sw   t0, 8(t1)      // B[j+2] = A[i-3] + A[i+3]
```

Quiz-2

```
mystery:  add $s2, $zero, $zero
          add $t1, $zero, $a0
          add $t3, $zero, $a1
apple:    slt $t2, $t1, $t3
          bne $t2, $zero, orange
          addi $t1, $t1, -1
          addi $t3, $t3, 2
          addi $s2, $s2, 2
          j apple
orange:   add $v0, $s2, $a1
          jr $ra
```

	$t1	$t2	$t3	$s2
Init	33		12	0
L1	…	…	…	…
…	…	…	…	…

- **We wrote a new MIPS function called "mystery" that takes two arguments as: void mystery(int a, int b). What is the value returned if it is called as mystery(33,12)? No justification, no credit.**
- 33 in $a0, 12 in $a1, return value in $v0

Quiz-2 Solution

```
mystery:   add $s2, $zero, $zero
           add $t1, $zero, $a0
           add $t3, $zero, $a1
apple:     slt $t2, $t1, $t3
           bne $t2, $zero, orange
           addi $t1, $t1, -1
           addi $t3, $t3, 2
           addi $s2, $s2, 2
           j apple
orange:    add $v0, $s2, $a1
           jr $ra
```

	$t1	$t2	$t3	$s2
Init	33		12	0
L1	32	0	14	2
L2	31	0	16	4
L3	30	0	18	6
Ln	33-n	0	12+2n	2n
L7	26	0	26	14
L8	25	0	28	16
L9		1		

$v0=16+12=28

- **We wrote a new MIPS function called "mystery" that takes two arguments as: void mystery(int a, int b). What is the value returned if it is called as mystery(33,12)? No justification, no credit.**
- 33 in $a0, 12 in $a1, return value in $v0

Quiz-3

- Assume that a branch instruction (conditional/unconditional) takes 2 cycles, a memory transaction takes 5 cycles; arithmetic operation takes 3 cycles. Given a 2GHz processor, what would be the **exact** CPU time in **milliseconds** to clear a string of length 1MB (**2^20 Bytes**)? Show your work.

```
/ Writes zeros to a byte string
// void bzero (void* b, size_t len)
bzero:    add $t0, $0, $a0
          add $t1, $0, $a1
loop:     beq $t1, $0, exit
          sb $0, 0($t0)
          addi $t1, $t1, -1
          addi $t0, $t0, 1
          j loop
exit:     jr $ra
```

Quiz-3 solution

- Assume that a branch instruction (conditional/unconditional) takes 2 cycles, a memory transaction takes 5 cycles; arithmetic operation takes 3 cycles. Given a 2GHz processor, what would be the **exact** CPU time in **milliseconds** to clear a string of length 1MB (**2^20 Bytes**)? Show your work.

```
/ Writes zeros to a byte string
// void bzero (void* b, size_t len)
bzero:      add $t0, $0, $a0(3)
            add $t1, $0, $a1(3)
loop:       beq $t1, $0, exit(2)
            sb $0, 0($t0)   (5)
            addi $t1, $t1, -1(3)
            addi $t0, $t0, 1(3)
            j loop          (2)
exit:       jr $ra          (2)
```

8 cycles out of loop

15 per loop, 2^20 loops

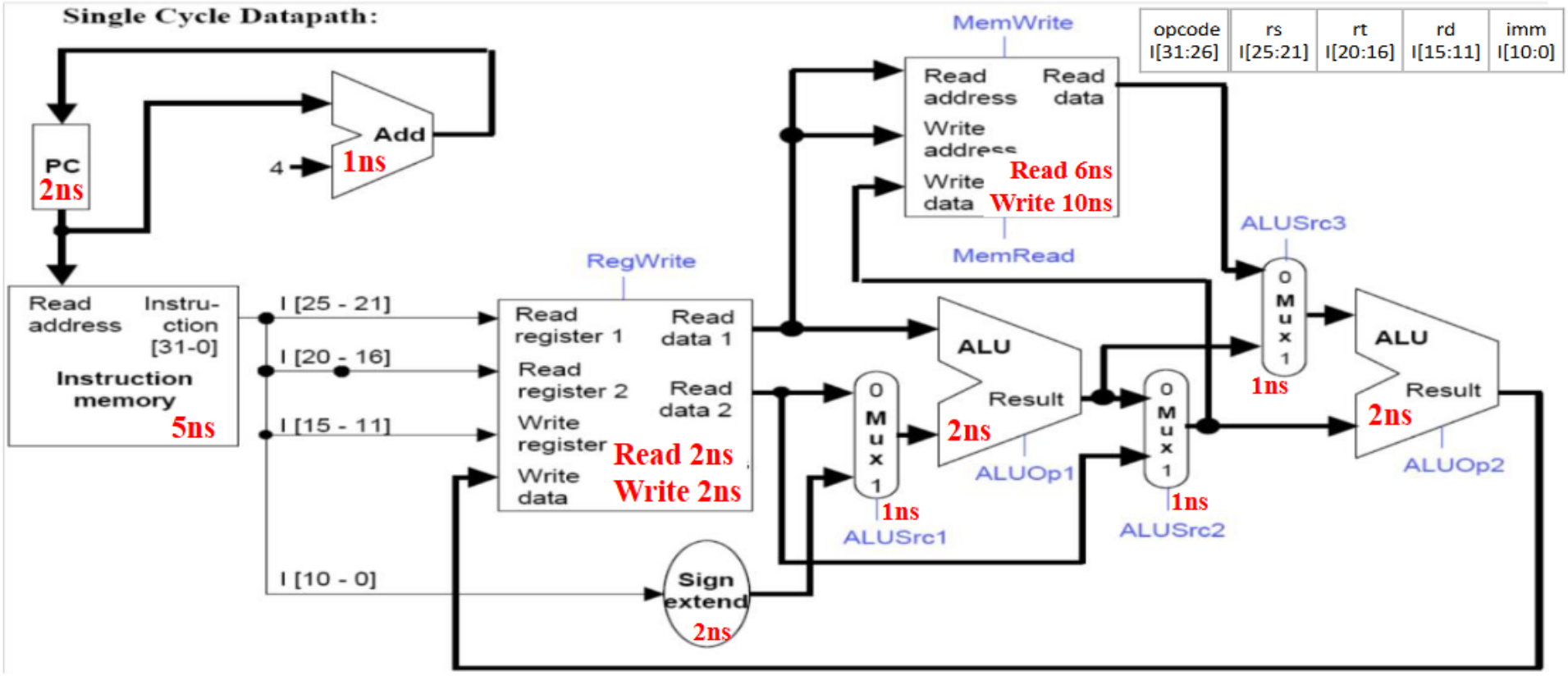
2 cycles for last branch

2^20*15 + 8+ 2 cycles
Cycle time 1/(2*10^9)
Execution time = Cycle count * Cycle time = 7.86ms

```
swi   rd,rs,rt,imm      # Memory[R[rs]] = R[rt],
                        # R[rd]=R[rs]+R[rt]+Imm
                        # PC = R[rs]+Imm
```

Indicate value of each control signal. You may indicate **ALU operation (ALUOp) as add, sub, mul, sll, and srl.** Use X for **don't care**. Modify the datapath if needed. Full credit will be given to the design with minimal hardware overhead. Refer to the delays given in the datapath and calculate the critical path delay for this instruction. Show your work and list all the components that are on the critical path.

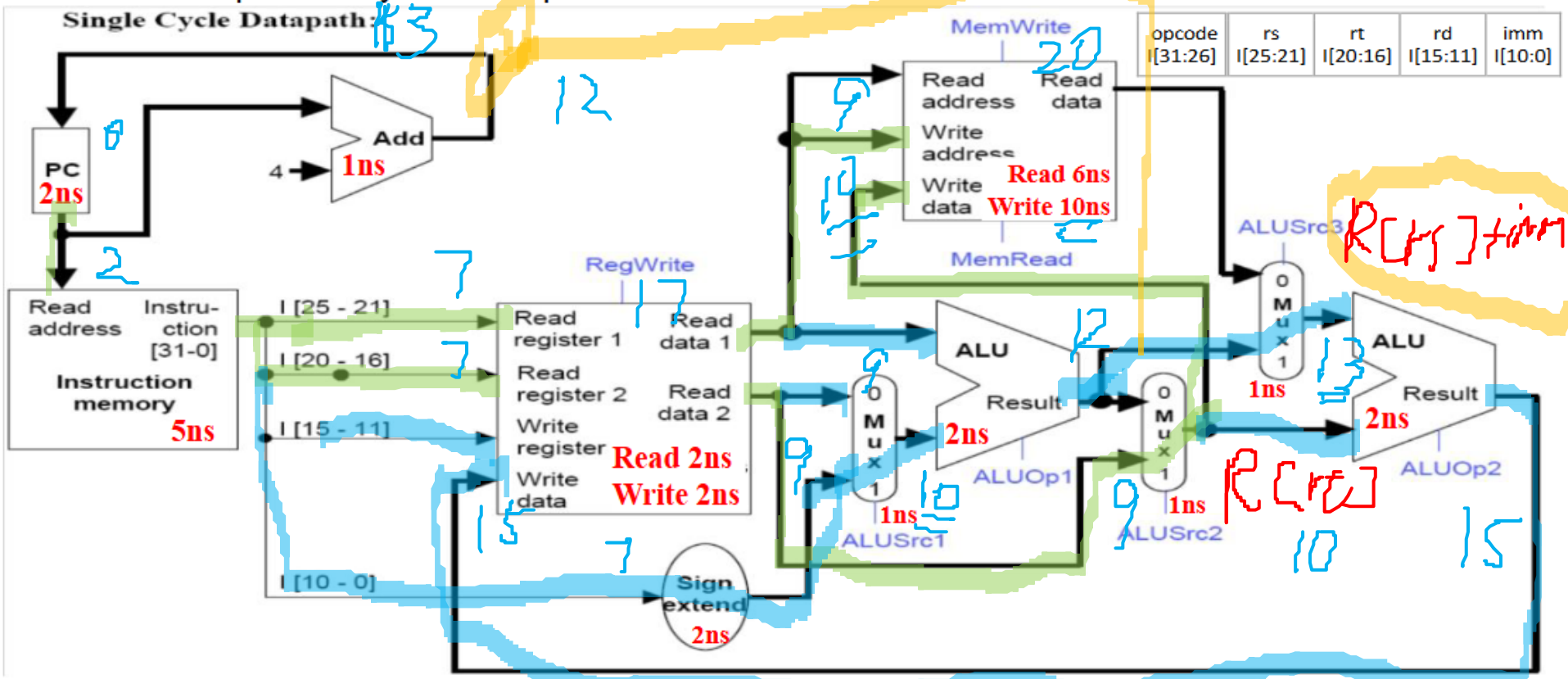
- Must show all **active** paths
  - **Show concurrency**
- Indicate path delay for each path
- Justify the longest path
- Label all nodes that are on the critical path
- Draw your wires and muxes clearly



```
swi   rd,rs,rt,imm      # Memory[R[rs]] = R[rt],
                        # R[rd]=R[rs]+R[rt]+Imm
                        # PC = R[rs]+Imm
```

Indicate value of each control signal. You may indicate **ALU operation (ALUOp) as add, sub, mul, sll, and srl.** Use X for **don't care**. Modify the datapath if needed. Full credit will be given to the design with minimal hardware overhead. Refer to the delays given in the datapath and calculate the critical path delay for this instruction. Show your work and list all the components that are on the critical path.

- Must show all **active** paths
  - **Show concurrency**
- Indicate path delay for each path
- Justify the longest path
- Label all nodes that are on the critical path
- Draw your wires and muxes clearly



Quiz 5– Which cycle will the last instruction be completed?

- Regular 5-stage pipeline, **no forwarding**
- Branch resolved in the MEM stage: next instruction can be fetched **1 cycle after** beq reaches the MEM stage **if branch is taken**
- Hint: loop runs for 40 times

```
        addi $t0, $t0, 160
Loop:   add  $t1, $a0, $t0
        lw   $t2, (0)$t1
        lw   $t3, (4)$t1
        addi $t0, $t0, -4
        bne  $t0, $zero, Loop
        addi $t3, $t0, 4
        sub  $t4, $t2, $t3
        and  $t5, $t2, $t3
```

Quiz 5– Solution

- Regular 5-stage pipeline, **no forwarding**
- Branch resolved in the MEM stage: next instruction can be fetched **1 cycle after** beq reaches the MEM stage **if branch is taken**
- Hint: loop runs for 40 times

```
        addi $t0, $t0, 160

Loop:   add  $t1, $a0, $t0
        lw   $t2, (0)$t1
        lw   $t3, (4)$t1
        addi $t0, $t0, -4
        bne  $t0, $zero, Loop
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
F	D	E	M	W											
			F	D	E	M	W								
						F	D	E	M	W					
							F	D	E	M	W				
								F	D	E	M	W			
											F	D	E	M	W
															F

477	478	479	480	481	482	483	484	485	486	487	488	489
F	D	E	M	W								
			F	D	E	M	W					
				F	D	E	M	W				
							F	D	E	M	W	
								F	D	E	M	W

```
        addi $t3, $t0, 4
        sub  $t4, $t2, $t3
        and  $t5, $t2, $t3
```